High-Performance Computing

MPhil Data Intensive Science

Barbara Koch

Word Count: 2237

April 1, 2026

Abstract

This coursework develops an in-place Cholesky factorization routine $C = LL^T$ in C for a single 76-core CSD3 icelake node, using OpenMP for shared-memory parallelism.

The implementation was built in stages. I began from the provided baseline algorithm (`v1`), then first improved single-thread performance through better memory access and compiler optimisation (`v2`). I next added a straightforward OpenMP parallelisation of the trailing update (`v3`). This improved performance at moderate thread counts, but the flat OpenMP version still synchronised once per column step.

To reduce this overhead, I rewrote the algorithm in blocked form (`v4`), processing panels of width NB = 96. This changed the synchronisation frequency from $O(n)$ to $O(n/\mathrm{NB})$ per factorisation, and strong scaling improved in the measurements. I then introduced several locality- and throughput-oriented optimisations in the final version (`v5`), including column packing, private per-thread panel storage, four-way unrolling of the trailing update, and round-robin scheduling in the trailing update phase.

Correctness was checked throughout using exact small examples, reconstruction residuals, output-layout checks, and agreement across thread counts. Relative to the `-O0` baseline, the serially optimised version improves single-thread performance by around 13–17×. The blocked OpenMP version `v4_openmp_blocked` reaches 147.2 GFLOP/s at 76 threads for $n = 8000$, corresponding to a speedup of 59.5× and a parallel efficiency of 78.2%. The final implementation, `v5_openmp_blocked` (tag `v0.5-tuned`), reaches 182.7 GFLOP/s at 76 threads for $n = 8000$, a 24% improvement over `v4_openmp_blocked` at the same thread count.

1 Introduction

Cholesky factorization is a standard tool in scientific computing whenever a symmetric positive-definite matrix must be inverted, solved against, or used to compute a determinant. In the context of covariance matrices, this is particularly important because the log-determinant can be recovered directly from the diagonal of the Cholesky factor,

$$\log |C| = 2 \sum_{p=0}^{n-1} \log L_{pp}. \tag{1}$$

For dense $n \times n$ matrices, the cost of Cholesky factorization is $O(n^3)$, so even moderate increases in efficiency can have a large effect on runtime. The main challenge in this coursework was therefore not only to produce a correct implementation, but to improve its performance in a controlled way and to justify each optimisation with measurements.

I approached the coursework as a sequence of implementation stages. First, I established a working baseline that follows the provided algorithm directly. Second, I improved its single-thread behaviour by changing loop order and reducing unnecessary memory traffic. Third, I parallelised the trailing update with OpenMP and measured how far this simple approach would scale. Finally, after observing that the flat OpenMP version scaled poorly at high thread counts, I restructured the algorithm into a blocked form and then optimised the blocked kernel further for cache locality, instruction-level parallelism, and load balance.

This report follows that development path. Rather than presenting only the final code, it explains what was changed at each stage, why the change was made, and how the measured results motivated the next step. The final implementation, `v5_openmp_blocked` (tag `v0.5-tuned`), achieves 182.7 GFLOP/s at 76 threads for $n = 8000$ on a single CSD3 icelake node.

In the reported data, throughput is higher at smaller problem sizes (for example 238.4 GFLOP/s at $n = 2000$, 76 threads) than at $n = 8000$ (182.7 GFLOP/s, 76 threads).

2 Algorithm and Interface

2.1 Cholesky recurrence

The factorisation proceeds column by column for $p = 0, \ldots, n-1$. At step p, after the contributions from columns $0, \ldots, p-1$ have already been subtracted from the working matrix, the diagonal entry is

$$L_{pp} = \sqrt{C_{pp}^{(p)}}, \tag{2}$$

and the entries below the diagonal are

$$L_{ip} = \frac{C_{ip}^{(p)}}{L_{pp}}, \qquad i > p. \tag{3}$$

The remaining trailing submatrix is then updated by

$$C_{ij}^{(p+1)} = C_{ij}^{(p)} - L_{ip} L_{jp}, \qquad i, j > p. \tag{4}$$

2.2 Function interface

The function

```
double mphil_dis_cholesky(double *c, int n);
```

takes a pointer to a row-major $n \times n$ matrix and performs the factorisation in place.

On return:

- the lower triangle contains L,
- the upper triangle contains L^T.

The function returns the wall-clock time in seconds. Inputs with $n < 1$ or $n > 100000$ are rejected.

The repository contains five main implementation versions of the routine, corresponding to successive optimisation stages. These stages are tracked with tags `v0.1-baseline` through `v0.5-tuned`. The implementation used for the main results is `cholesky_v5_openmp_blocked.c` (tag `v0.5-tuned`); the earlier tagged versions are retained as intermediate development stages.

2.3 Correctness verification

Correctness was assessed using several complementary checks. The main criterion is the reconstruction error of the factorisation, rather than the log-determinant alone.

The test suite was run on a CSD3 icelake node for both:

- a **strict build** (without `-ffast-math`),
- a **performance build** (with `-ffast-math`),

and across multiple thread counts ($T = 1, 2, 4, 8, 76$). The final correctness run passed in all tested configurations.

1. **Known-L tests.** Matrices of the form $C = L_{\text{ref}} L_{\text{ref}}^T$ were generated, and the computed factor was compared directly to L_{ref}. Test sizes included values around block boundaries ($n = 95, 96, 97$, etc.), where indexing errors are more likely.

   The maximum observed errors were small and consistent with double precision accuracy.

2. **Output layout.** The symmetry between the lower and upper triangles was checked in the test suite.

3. **Boundary cases.** The implementation correctly handles $n = 1$ and rejects invalid sizes.

4. **Thread consistency.** Results obtained with multiple threads were compared to the single-thread result. The maximum observed differences were within tolerance ($\max |\Delta| < 10^{-10}$).

The log-determinant was used as a secondary check only. While it provides a useful scalar summary, it does not detect all possible errors (for example, incorrect off-diagonal entries).

3 Optimisation Strategy

3.1 v1 — Baseline (tag `v0.1-baseline`)

The baseline implementation follows the provided algorithm and is compiled with `-O0`. The trailing update uses a loop ordering where `c[i*n+j]` is accessed with a large stride in the inner loop. For large n, this is expected to limit cache efficiency.

Snippet 1 shows the baseline trailing update. Because the matrix is stored in row-major order, the inner loop over i walks memory with stride n, which gives poor spatial locality.

Listing 1: Baseline trailing-update kernel in v1.

```
for (int j = p + 1; j < n; j++) {
    for (int i = p + 1; i < n; i++) {
        c[i*n + j] -= c[i*n + p] * c[p*n + j];
    }
}
```

3.2 v2 — Serial optimisation (tag `v0.2-serial-opt`)

Two simple changes are made to improve single-thread performance:

- **Loop interchange.** The inner loop is changed to iterate over j, making `c[i*n+j]` contiguous in memory. This is intended to improve cache locality and may allow the compiler to vectorise the loop more effectively.
- **Loop-invariant hoisting.** The value `c[i*n+p]` is reused inside the inner loop, so it is loaded once per row instead of once per iteration.

Snippet 2 shows the optimised trailing-update kernel. The outer loop now walks rows, while the inner loop sweeps contiguous elements within each row.

Listing 2: Optimised trailing-update kernel in v2.

```
for (int i = p + 1; i < n; i++) {
    double c_ip = c[i*n + p];
    for (int j = p + 1; j < n; j++) {
        c[i*n + j] -= c_ip * c[p*n + j];
    }
}
```

With compiler optimisations enabled (`-O3 -march=native -ffast-math`), this version is faster than the baseline in the measurements.

3.3 v3_openmp — Flat OpenMP parallelism (tag `v0.3-openmp-v1`)

The trailing update is parallelised over rows using `#pragma omp for`. Since each row is independent, this is a natural choice for parallelisation.

A single `omp parallel` region is used around the outer loop to avoid repeated thread creation. Within each iteration, `omp single` is used for the diagonal and normalisation steps, followed by a barrier before the parallel update.

Snippet 3 shows the flat OpenMP structure. The key limitation is that the `single` and `for` regions are encountered once per column, so the algorithm still synchronises at every step.

Listing 3: Flat OpenMP structure in v3.

```
#pragma omp parallel default(none) shared(c, n)
{
    for (int p = 0; p < n; p++) {
        #pragma omp single
        { /* diagonal update and normalisation */ }

        #pragma omp for schedule(static)
        for (int i = p + 1; i < n; i++) {
            /* update row i of the trailing submatrix */
        }
    }
}
```

This approach improves performance at moderate thread counts. However, for $n = 2000$, performance peaks at around 48 threads and then decreases at higher thread counts.

3.4 v4_openmp_blocked — Panel-blocked algorithm (tag `v0.4-blocked`)

To reduce synchronisation overhead, the algorithm is restructured to process blocks of columns instead of a single column at a time.

The matrix is processed in panels of width NB. For each panel, three steps are performed:

- **Phase 1 (serial):** factorise the diagonal block.
- **Phase 2 (parallel):** solve for the block below the panel.
- **Phase 3 (parallel):** update the trailing submatrix.

Snippet 4 shows the blocked outer structure. One synchronisation sequence now covers an entire panel rather than a single column.

Listing 4: Panel-blocked structure in v4.

```
#pragma omp parallel default(none) shared(c, n)
{
    for (int k = 0; k < n; k += BLOCK_NB) {
        int kend = (k + BLOCK_NB < n) ? (k + BLOCK_NB) : n;

        #pragma omp single
        { /* Phase 1: factor diagonal block */ }

        #pragma omp for schedule(static)
        for (int i = kend; i < n; i++) {
            /* Phase 2: TRSM below the panel */
        }

        #pragma omp for schedule(guided)
        for (int i = kend; i < n; i++) {
            /* Phase 3: SYRK trailing update */
        }
    }
}
```

This reduces the number of global synchronisation points from $O(n)$ to $O(n/\mathrm{NB})$, which is expected to improve scalability.

The panel width was chosen empirically. Values in the range NB = 64–256 were tested. In the committed block-sweep results at $n = 8000$, NB = 256 is fastest at 8 and 32 threads. At 76 threads the differences between NB values are small and the measurements are noisier. NB = 96 is used for the main scaling runs.

3.5 v5_openmp_blocked — Further optimisation (tag `v0.5-tuned`)

Several additional optimisations are introduced on top of v4. These aim to improve memory access patterns and increase instruction-level parallelism.

- **Column packing (Phase 1).** The column used in the panel update is copied into a contiguous buffer. This is intended to avoid repeated long-stride accesses.

- **Per-thread packed panel (Phase 2).** Each thread copies the current panel block into a private buffer. This is intended to reduce repeated access to widely spaced memory locations during the triangular solve.
- **Loop unrolling (Phase 3).** The inner update loop is unrolled to process multiple columns at once. This is intended to increase the amount of independent work available to the processor.
- **Scheduling strategy.** `schedule(static,1)` is used in the trailing update to distribute work more evenly across threads, as the amount of work per row increases with the row index.

Snippet 5 illustrates the most performance-critical part of the final kernel: the trailing update is assigned round-robin across threads and the inner work is unrolled to accumulate four dot products at once.

Listing 5: Round-robin scheduling and four-way unrolling in v5.

```
#pragma omp for schedule(static, 1)
for (int i = kend; i < n; i++) {
    for (j = kend; j <= i - 3; j += 4) {
        double d0 = 0.0, d1 = 0.0, d2 = 0.0, d3 = 0.0;
        #pragma omp simd reduction(+:d0,d1,d2,d3)
        for (int p = 0; p < panel_width; p++) {
            double pi = panel_i[p];
            d0 += pi * pj0[p]; d1 += pi * pj1[p];
            d2 += pi * pj2[p]; d3 += pi * pj3[p];
        }
    }
}
```

These changes are associated with higher measured performance, although their individual contributions are not isolated in this report. The final version shows higher measured performance than v4 at both 1 and 76 threads.

4 Experimental Setup

Platform. All experiments were run on a single node of the CSD3 icelake partition. Jobs were submitted using SLURM, typically requesting one full node.

Compilation. The following compiler settings were used:

Version	Flags
v1	`-O0 -g`
v2	`-O3 -march=native -ffast-math`
v3–v5	`-O3 -march=native -ffast-math -fopenmp`

All versions were compiled with `gcc` (module `gcc/11`).

Runtime settings. OpenMP threads were pinned to cores using `OMP_PROC_BIND=close` and `OMP_PLACES=cores`. The number of threads T was varied through `OMP_NUM_THREADS`.

Methodology. Matrix sizes $n \in \{2000, 4000, 6000, 8000\}$ were used to cover a range of problem sizes.

Thread counts $T \in \{1, 2, 4, 8, 16, 32, 48, 64, 76\}$ were used to study strong scaling from a single core to the full node.

Each configuration was run three times. In the committed scaling CSV, the blocked $n = 8000$ points were also measured in a second SLURM job, giving six timing samples for those points. Means and standard deviations are computed over all available samples. For `v5_openmp_blocked` at $n = 8000$, the two jobs produced noticeably different timings at $T = 8$, which is visible as a larger error bar at that point in the scaling figures.

Performance is reported in GFLOP/s using the leading-order Cholesky cost,

$$\text{GFLOP/s} = \frac{n^3}{3t \times 10^9},$$

where t is the measured runtime. All timings include only the factorisation routine; setup and validation are excluded.

The benchmark program records the actual OpenMP thread count at runtime, so reported values reflect the effective number of threads used.

All figures in the report are generated from CSV data in `results/` using the provided plotting scripts.

Git tags. The development of the code is tracked using tagged commits, corresponding to the main optimisation stages:

Tag	File	Description
`v0.1-baseline`	`cholesky_v1_baseline.c`	Baseline implementation at `-O0`
`v0.2-serial-opt`	`cholesky_v2_serial_opt.c`	Improved memory access and compiler optimisations
`v0.3-openmp-v1`	`cholesky_v3_openmp.c`	First OpenMP version (parallel trailing update)
`v0.4-blocked`	`cholesky_v4_openmp_blocked.c`	Panel-blocked algorithm
`v0.5-tuned`	`cholesky_v5_openmp_blocked.c`	Final version with improved locality and scheduling

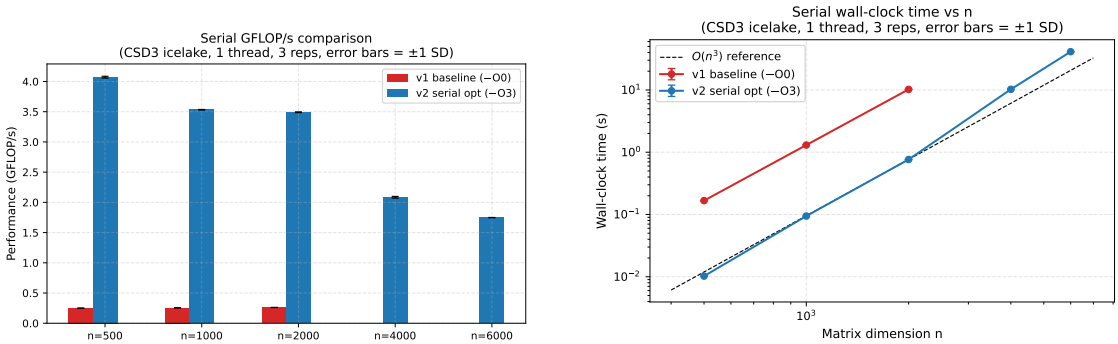
Each tag corresponds to a working version of the code and can be inspected directly in the repository.

5 Results

5.1 Serial optimisation

Figure 1a shows the effect of the serial optimisations. Version v2 is around 13–17× faster than v1 across $n = 500$–2000.

Both versions follow the similar slope in the log–log time plot (Figure 1b).



(a) GFLOP/s for v1 and v2 (1 thread).

(b) Runtime vs. n (log–log scale).

Figure 1: Serial performance on CSD3 icelake. Mean ±1 SD over 3 runs.

5.2 Incremental optimisation

Figure 2 shows single-thread performance at each stage.

The transition from v1 to v2 gives the largest improvement. The v2 and v3 results are similar at 1 thread. The v5 result is higher than the v4 result at 1 thread.

5.3 Panel-width selection

Figure 3 shows the effect of panel width for `v5_openmp_blocked` at $n = 8000$.

The best measured panel width depends on thread count in the committed sweep. NB=256 is fastest at 8 and 32 threads. At 76 threads the differences between NB values are small and the measurements are noisier. NB=96 is used in the remainder of the study.

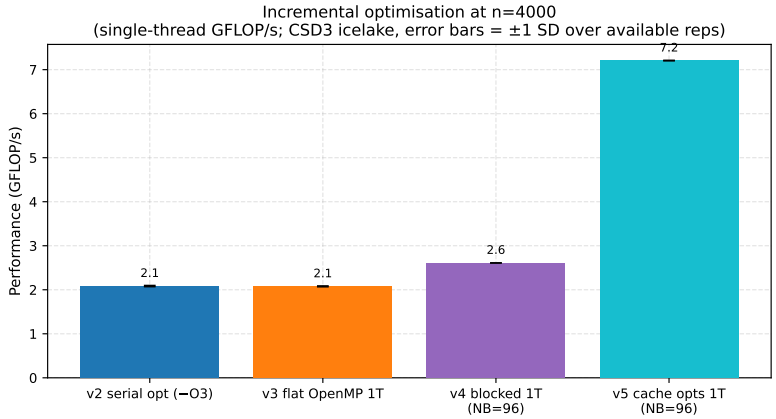


Figure 2: Single-thread GFLOP/s at $n = 4000$ for v2–v5.

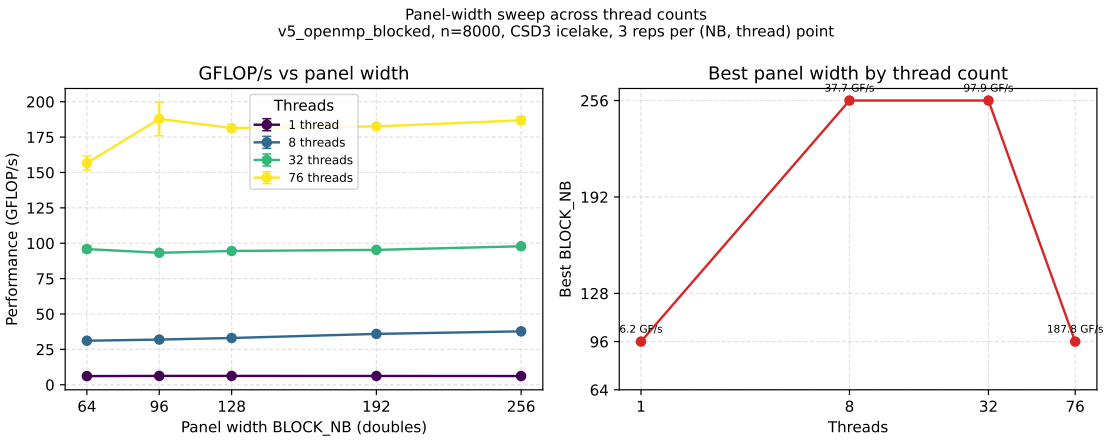


Figure 3: Panel-width sweep for `v5_openmp_blocked` at $n = 8000$.

5.4 Strong scaling

Figures 4–6 show strong scaling for v3, v4, and v5.

The v3 implementation improves with thread count initially, but does not scale well to the full node. At high thread counts the marginal gains become small.

In contrast, both blocked versions (v4 and v5) scale more consistently. For $n = 8000$, v4 reaches 147.2 GFLOP/s at 76 threads, while v5 reaches 182.7 GFLOP/s.

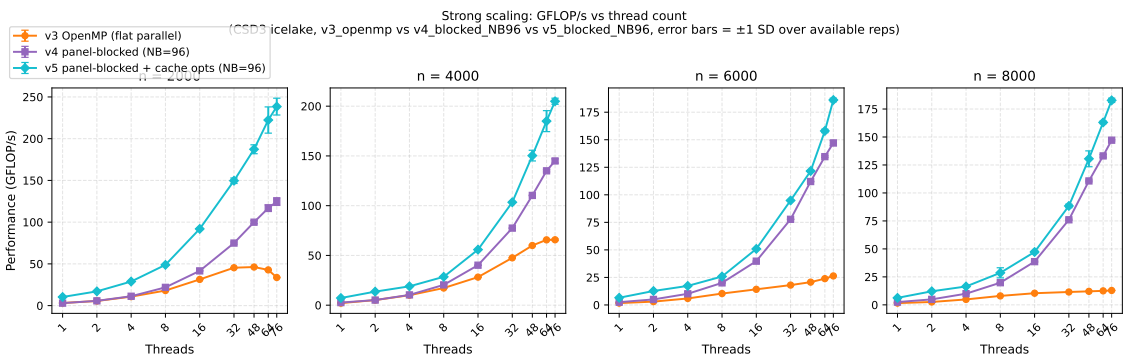


Figure 4: GFLOP/s vs. thread count for v3, v4, and v5.

5.5 Summary at $n = 8000$, 76 threads

At $n = 8000$, v3 reaches 12.9 GFLOP/s at 76 threads, compared with 147.2 GFLOP/s for v4 and 182.7 GFLOP/s for v5. The lower efficiency of v5 relative to v4 is consistent with its higher

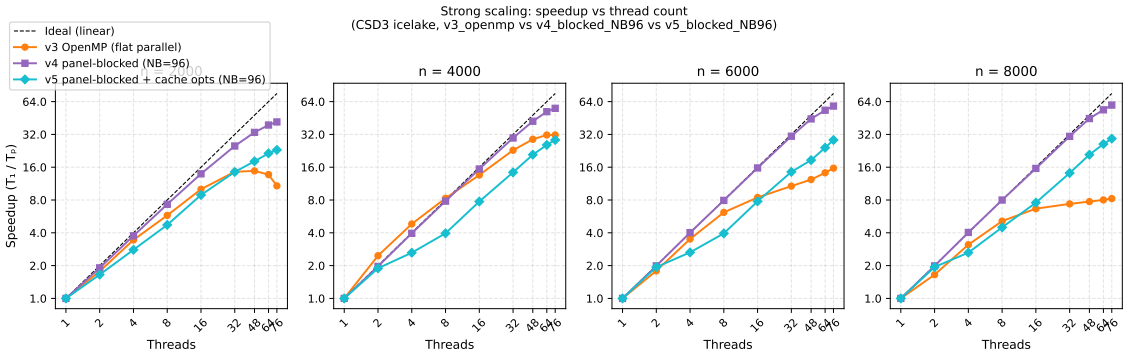


Figure 5: Speedup relative to 1 thread.

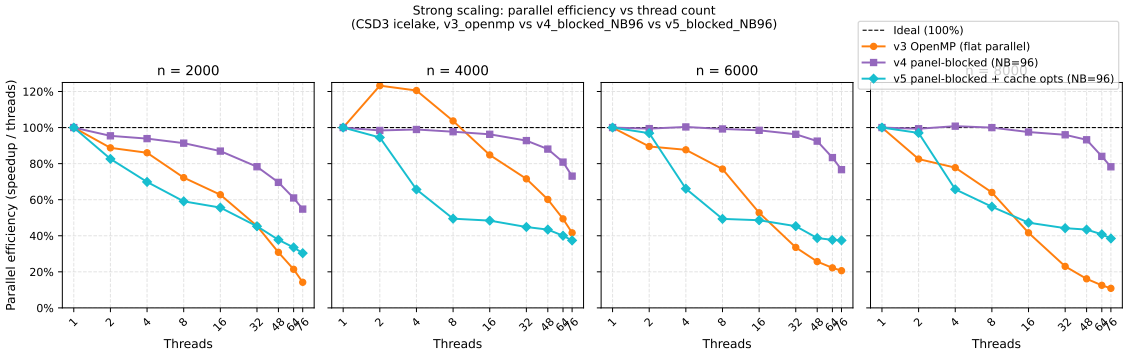


Figure 6: Parallel efficiency vs. thread count.

1-thread baseline.

5.6 Head-to-head comparison

Figure 7 compares the three implementations at $n = 8000$.

The difference between the flat and blocked approaches is substantial in the measurements. The blocked versions outperform v3 across all measured thread counts. The improvement from v4 to v5 is smaller but consistent.

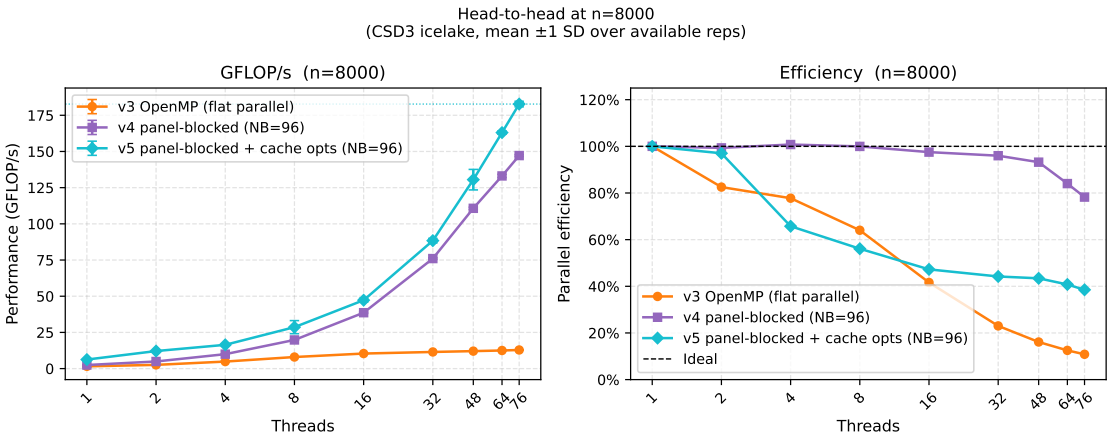


Figure 7: Comparison of v3, v4, and v5 at $n = 8000$.

5.7 Problem-size scaling

Figure 8 shows performance of v5 as a function of n.

Performance is highest at smaller sizes and decreases as n increases. This trend is consistent across most thread counts.

Table 1: Performance at $n = 8000$, 76 threads.

Version	GFLOP/s	Speedup	Efficiency
v3_openmp	12.9	8.2×	10.8%
v4_openmp_blocked	147.2	59.5×	78.2%
v5_openmp_blocked	182.7	29.3×	38.5%

At 76 threads, performance decreases from 238.4 GFLOP/s at $n = 2000$ to 182.7 GFLOP/s at $n = 8000$.

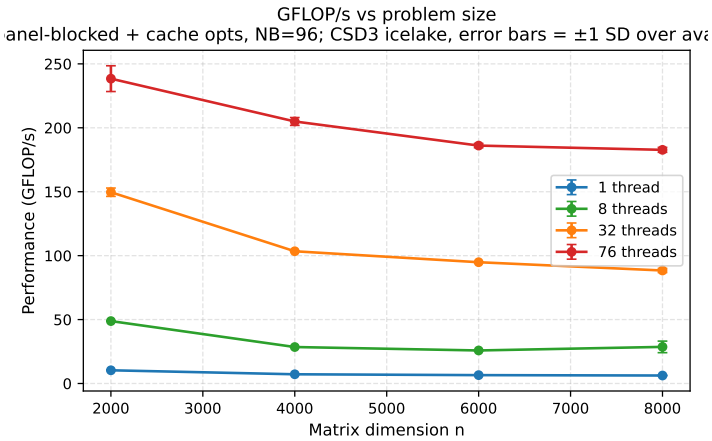


Figure 8: GFLOP/s vs. problem size for v5.

6 Discussion

Scaling behaviour. The committed performance data shows three main patterns. First, the flat OpenMP version improves over its 1-thread baseline but remains well below the blocked versions at high thread counts. Second, the blocked versions extend scaling further, with `v5_openmp_blocked` giving the highest reported performance in the main $n = 8000$ comparison. Third, for `v5_openmp_blocked`, the reported GFLOP/s at 76 threads decreases as n increases from 2000 to 8000.

7 Conclusion

Starting from a simple baseline implementation, a series of optimisations were applied to improve both single-thread and multi-thread performance.

Basic compiler optimisations and loop restructuring provided a large improvement on a single core. Introducing OpenMP parallelism gave further gains, although scaling was more limited in the flat version than in the blocked versions in the reported runs.

Rewriting the algorithm in a blocked form improved scalability by reducing the number of synchronisation points. Additional changes aimed at improving data locality and loop structure further increased performance.

The final implementation (`v5_openmp_blocked`) reaches 182.7 GFLOP/s at 76 threads for $n = 8000$ on a CSD3 icelake node.